

3 LLM-Guided Automatic Heuristic Design

We solve the empirical code-optimization problem using an automatic heuristic design procedure inspired by Evolution of Heuristics (EoH) (Liu et al., 2024). The key difficulty is that the decision variable is an executable policy rather than a continuous vector. A candidate policy cannot be improved by taking a gradient with respect to its code; it can only be evaluated by simulating its ordering decisions on the training demand trajectories. We therefore treat policy search as a population-based black-box optimization problem over executable stationary policies.

3.1 Overview of the Search Procedure

The algorithm maintains a surviving pool of candidate policies. Each candidate is an executable policy π together with its empirical training cost $\hat{J}(\pi)$. Let K denote the maximum size of the surviving pool, M the number of new candidate policies generated in each generation, m the number of parent policies supplied to the LLM prompt, and G the number of generations.

The search proceeds in two stages. First, the algorithm constructs an initial candidate pool from a base-stock policy. This initial policy is checked for executability, optionally passed through a numerical parameter optimizer, evaluated on the training trajectories, and then used to form the first surviving pool $\mathcal{P}^{(0)}$.

After initialization, the algorithm repeats an evolutionary loop. At generation g , it selects a parent set $\mathcal{A}^{(g)} \subseteq \mathcal{P}^{(g)}$ of size m from the current surviving pool. The selected parents are inserted into a modification prompt together with the problem statement, the policy interface, the performance statistics of the parents, and validity rules for generated code. The LLM then proposes M new candidate policies. Each candidate is again validated, optionally parameter-tuned, and evaluated on the training trajectories. The new candidates are merged with the current surviving pool, and the best K feasible policies are retained to form $\mathcal{P}^{(g+1)}$. After G generations, the algorithm returns the policy with the lowest empirical training cost.

This overview is intentionally high level. The central idea is simple: start from a feasible policy pool, repeatedly ask the LLM to generate improved policy code from high-performing parents, use simulation to score the generated policies, and keep only the best survivors. The remainder of this section describes each module in the order in which it appears in the algorithm.

3.2 Policy Code and Feasibility

Each candidate policy is represented by executable code implementing a stationary decision rule

$$a_t = \pi(I_t, Q_t),$$

where (I_t, Q_t) is the inventory state defined in the model section. The code receives the current state information and returns a nonnegative order quantity. Only the executable policy enters the search pool; auxiliary text produced by the LLM is not used as part of the candidate representation.

A generated policy is feasible only if it can be parsed, executed, and evaluated in the simulator. It must return a valid order quantity and must satisfy the stationarity requirement: the order quantity may depend on the current inventory state, but not on hidden memory, episode index, time counters, or information from previous calls. This restriction is important because the goal is to discover deployable stationary ordering logic, not a program that exploits details of the simulation protocol.

3.3 Initialization

The first module creates the initial candidate pool. The implementation starts from the standard base-stock policy

$$\pi_B(I_t, Q_t) = \max \left\{ 0, B - I_t - \sum_{k=1}^L q_{t,k} \right\},$$

where B is the base-stock level. This initializer gives the search a stable and interpretable starting point. It also ensures that the first surviving pool contains a valid policy whose behavior is familiar in inventory control.

The initial base-stock candidate then passes through the same downstream pipeline: code validation, optional parameter optimization, empirical evaluation, and survival filtering. Thus, initialization does not simply load a baseline; it creates the first evaluated and filtered surviving pool $\mathcal{P}^{(0)}$.

3.4 Parameter Optimization

Many inventory policies contain numerical thresholds or scaling constants, such as base-stock levels, safety-stock buffers, pipeline weights, or order caps. These constants can strongly affect performance, and they are difficult for an LLM to tune precisely. To reduce the burden on the LLM, we separate structural search from numerical tuning.

When the prompt asks the LLM to generate policy code, it also explains how to mark tunable constants using an `OPT_PARAM` annotation. A marked parameter specifies an initial value, lower and upper bounds, and a type. For a generated policy structure s , let θ_s be the vector of marked constants and let Θ_s be the corresponding bounded search region. If the generated code contains marked parameters and the external optimizer is enabled, the algorithm solves the inner tuning problem

$$\theta_s^* \in \arg \min_{\theta \in \Theta_s} \hat{J}(\pi_s, \theta).$$

The code structure is fixed during this step; only the marked numerical constants are changed. If a policy has no marked constants, or if the external optimizer is disabled, the policy is evaluated as generated. In implementation, the unoptimized version is also evaluated, and the algorithm keeps the better of the original and tuned versions. This prevents a useful policy structure from being discarded only because its initial constants were poorly chosen.

3.5 Empirical Evaluation

After validation and optional parameter tuning, each candidate policy is evaluated by simulation on the training demand trajectories. For policy π , the empirical score is

$$\hat{J}(\pi) = \frac{1}{N} \sum_{n=1}^N C_n(\pi),$$

where $C_n(\pi)$ is the cumulative cost of policy π on trajectory n , as defined earlier. Lower values of $\hat{J}(\pi)$ indicate better training performance.

Invalid policies are excluded at this stage. A policy is treated as invalid if the code cannot be parsed, if execution fails, if it returns an invalid order quantity, if it violates the stationarity

restrictions, or if the evaluation times out. The surviving pool therefore contains only policies that can actually be simulated under the inventory dynamics.

3.6 Survival Selection

Once a batch of candidates has been evaluated, the algorithm applies survival selection. Let $\mathcal{C}^{(g)}$ denote the set of feasible candidates generated at generation g . The candidates are merged with the current surviving pool:

$$\mathcal{U}^{(g)} = \mathcal{P}^{(g)} \cup \mathcal{C}^{(g)}.$$

The algorithm then removes duplicate candidates according to their empirical objective values and retains the K unique policies with the lowest training costs:

$$\mathcal{P}^{(g+1)} = \text{TopK}(\mathcal{U}^{(g)}, \hat{J}, K).$$

This elitist update ensures that strong policies are not lost when a generated batch is poor, while still allowing newly generated policies to enter the pool when they improve on existing candidates.

3.7 Parent Selection

After the surviving pool has been updated, the next generation begins by selecting parents from the current pool. Let

$$\mathcal{P}^{(g)} = \{z_1^{(g)}, \dots, z_{K_g}^{(g)}\}$$

be the surviving pool at the start of generation g , where each $z_i^{(g)}$ corresponds to an executable policy and its empirical cost. The parent set is

$$\mathcal{A}^{(g)} = \{z_{(1)}^{(g)}, \dots, z_{(m)}^{(g)}\} \subseteq \mathcal{P}^{(g)}.$$

Here m is a user-specified number of parents supplied to the LLM. The selected parents are high-performing candidates from the surviving pool. The notation covers both single-parent and multi-parent prompting without treating them as different algorithms: the LLM always receives a set of parents, and m controls the size of that set.

3.8 Evolutionary Prompt and Candidate Generation

The evolutionary prompt is the main mechanism for generating new policies after initialization. Its purpose is to ask the LLM to use the selected parent set $\mathcal{A}^{(g)}$ as a starting point and produce new executable policies that may reduce the empirical training cost.

The prompt contains five components. First, the *task block* states the inventory problem, the planning and selling phases, the lost-sales cost structure, and the empirical objective. Second, the *interface block* specifies the policy function name, the input variables, the output variable, and the requirement that the output be a nonnegative order quantity. Third, the *parent block* provides the code and empirical cost statistics of the selected parents in $\mathcal{A}^{(g)}$. Fourth, the *generation instruction* asks the LLM to modify, combine, or refine the parent policies to produce a new policy expected to perform better on the training trajectories. Finally, the *validity block* states the stationarity restrictions, disallows hidden memory or time-dependent behavior, and explains how to mark tunable constants for the external optimizer.

The prompt does not need to reveal every implementation detail of the simulator. Its role is to communicate the policy interface, the performance signal, and the design constraints clearly enough for the LLM to propose executable ordering logic. The resulting code is then handled by the same validation, tuning, evaluation, and survival-selection modules described above. In each generation, this process is repeated until a batch of M candidate policies has been generated or attempted.

3.9 Complete Algorithm

The complete procedure can be summarized as follows.

1. **Initial candidate generation.** Create the initial candidate pool using the base-stock initializer.
2. **Initial tuning and evaluation.** Validate the initial base-stock candidate, optionally tune its marked parameters, and evaluate its empirical cost on the training trajectories.
3. **Initial survival selection.** Use the feasible initialized policy to form $\mathcal{P}^{(0)}$.
4. **Parent selection.** At generation g , select a parent set $\mathcal{A}^{(g)}$ of size m from the current surviving pool.
5. **Candidate generation.** Use the evolutionary prompt to generate a batch of M new candidate policies from the selected parents.
6. **Offspring tuning and evaluation.** Validate each generated candidate, optionally tune its marked parameters, and compute its empirical training cost.
7. **Population update.** Merge feasible offspring with the current surviving pool and retain the best K unique policies.
8. **Iteration and output.** Repeat Steps 4–7 for G generations and return the feasible policy with the lowest empirical training cost.

References

Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., and Zhang, Q. (2024). Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Proceedings of the 41st International Conference on Machine Learning*.